Lecture 5: Linear and Polynomial Regression Through a Fitness-Tracker Problem

AI 501: Mathematics for Artificial Intelligence

Fall 2026

Abstract

A fitness-tracker company wants to predict how many hours a person exercises each week. We follow this one problem through the whole lecture. We start by drawing a line through four data points, ask what "best line" should mean, turn that meaning into a loss function, and solve the resulting optimization problem in closed form with matrix calculus. We then add a second feature, bend the line into a polynomial, and watch what happens when the model becomes too flexible. Every formula is checked on numbers small enough to compute by hand.

Contents

1 One problem: predicting exercise from age **2**
- 1.1 The intuition: draw a line . . . 2
- 1.2 Process, noise, and model . . . 3
- 1.3 The recipe . . . 3

2 The linear model **4**
- 2.1 What "linear" looks like . . . 4
- 2.2 Stacking the data into a matrix . . . 4
- 2.3 Many lines: what does "best" mean? . . . 5

3 The loss function **5**
- 3.1 From residuals to a loss . . . 5
- 3.2 The same loss in vector form . . . 6

4 The closed-form solution **7**
- 4.1 The intuition: a bowl . . . 7
- 4.2 Derivation . . . 8
- 4.3 Worked example: the age data . . . 9
- 4.4 What the answer means geometrically . . . 10
- 4.5 Why it is the minimum, and when the inverse exists . . . 10

5 More features: age and sleep **11**

6 Polynomial regression: bending the line **12**
- 6.1 When a line is not enough . . . 13
- 6.2 The trick: new features from old . . . 13
- 6.3 Our age data with $M = 0, 1, 2, 3$ . . . 14
- 6.4 A bigger example from the lecture: fitting a sine wave . . . 15

7 Underfitting and overfitting **16**
7.1 Two ways to be wrong . . . 16
7.2 Remedy 1: validate on held-out data . . . 16
7.3 Remedy 2: more data . . . 17
7.4 Remedy 3: regularization (next topic) . . . 17

8 The closed form in practice **18**
8.1 Do not compute the inverse . . . 18
8.2 Scale your features . . . 18
8.3 Cost . . . 18
8.4 A real-world problem: remaining useful life of a battery . . . 18
8.5 Where this leads . . . 19

9 Summary map **19**

1 One problem: predicting exercise from age

Where this fits. This section sets the scene and the vocabulary. Everything after it is an answer to the three questions at the end of Section 1.1.

A fitness-tracker company has logged the weekly exercise of four customers.

Age x (years)	Exercise y (hours per week)
25	4
30	5
40	3
50	2

The company wants a rule that takes a new customer's age and returns a predicted number of exercise hours. Because the predicted quantity is a number on a continuous scale (3.6 hours is a perfectly good answer), this kind of problem is called *regression*. Predicting a category, such as "will this customer cancel: yes or no?", is a different problem called classification.

1.1 The intuition: draw a line

Plot the four points (Figure 2 in the next section does exactly this). Older customers exercise less, roughly speaking, so a downward-sloping line looks reasonable. Once we have a line, prediction is easy: find the customer's age on the horizontal axis, go up to the line, and read off the height. Three questions follow immediately.

1. Many lines are possible. Which one is "best", and what does "best" mean? (Sections 2–4)
2. Age is not the only thing that matters. How do we use several inputs at once? (Section 5)
3. What if the points follow a curve rather than a line? (Sections 6–7)

The rest of the lecture answers these in order.

1.2 Process, noise, and model

Take two customers who are both 40 years old. One exercises 3 hours a week and the other 5. Age cannot explain the difference: job, injuries, motivation, and a hundred other things we did not record are at work. Regression handles this by assuming that the world works like this:

$$\boxed{y = f(\mathbf{x}) + n}$$

Here $\mathbf{x}$ is the *input* (the list of features we measured), f is an *unknown* function that captures the genuine relationship between input and output, n is *noise* (everything we did not measure), and y is the *observed output*. We never see f. We see only noisy samples $(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_n, y_n)$, called the *training data*. The goal is to estimate f from them as accurately as possible.

Figure 1 shows the idea. The straight line is the hidden f, the dots are what the company actually records, and the vertical scatter of the dots around the line is the noise. Read the figure backwards and you have the task: given only the dots, recover the line.

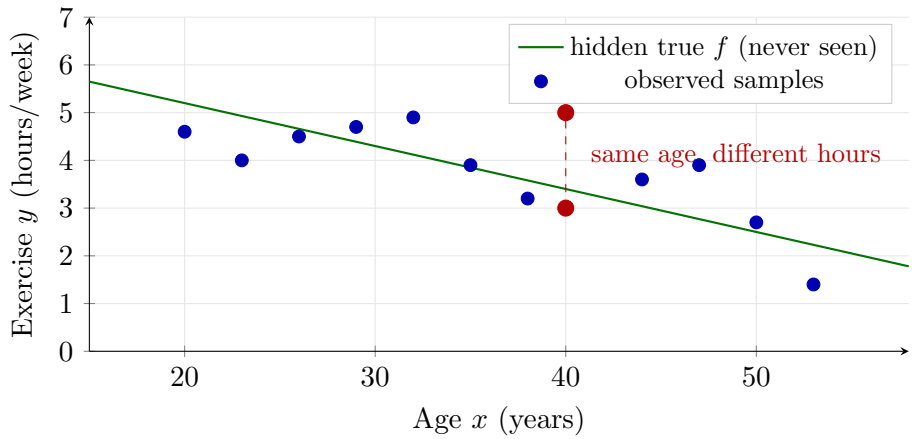


Figure 1: The regression set-up, with illustrative made-up data (not our four customers). Observations scatter around an unknown function; two customers of the same age can differ by hours.

To do this we choose a *model* $\hat{f}(\mathbf{x}, \mathbf{w})$ that has two ingredients:

- a *structure* that we pick in advance (a line, a polynomial, ...), and
- *parameters*, collected in a vector $\mathbf{w}$, that are tuned using the data.

For an input $\mathbf{x}$ the model outputs the *prediction* $\hat{y} = \hat{f}(\mathbf{x}, \mathbf{w})$, and the *error* (or *residual*) is $e = y - \hat{y}$. A hat always means "estimated by the model".

Notation. The slides call the parameters $\theta_0, \theta_1, \ldots$ in the general model and $w_0, w_1, \ldots$ in the worked example; they are the same numbers. These notes use $\mathbf{w}$ throughout, with w_0 the intercept and $w_1, \ldots, w_d$ the slopes.

1.3 The recipe

Linear regression is the second learning algorithm in the course. It differs from k-nearest neighbours in being *modular*. Almost everything we study later follows the same four steps, so it is worth memorizing them now.

The modular recipe. (1) Select a model. (2) Define a loss function that measures how badly the model fits the data. (3) Write the optimization problem "find the parameters that minimize the loss". (4) Solve it, either analytically (set the derivative to zero) or numerically (an algorithm that moves step by step towards the minimum).

This lecture does steps 1–4 with the analytical solution. The numerical alternative is the next topic.

2 The linear model

Where this fits. Step 1 of the recipe: choose the model, and arrange the data in the matrix form that the solution will need.

2.1 What "linear" looks like

With one feature (age), the model is a line,

$$\hat{y} = w_0 + w_1 x.$$

The intercept w_0 is the prediction at $x = 0$, and the slope w_1 is the change in prediction for each extra year. With two features x_1, x_2 the model is a plane $\hat{y} = w_0 + w_1 x_1 + w_2 x_2$, and with d features it is a *hyperplane* (a flat surface sitting in $d+1$ dimensions: d for the inputs and one more for the output),

$$\hat{y} = w_0 + \sum_{i=1}^{d} w_i x_i = w_0 + \mathbf{w}_{1:d}^{\mathsf{T}} \mathbf{x},$$

where $\mathbf{w}_{1:d} = (w_1, \ldots, w_d)^{\mathsf{T}}$ collects the slopes. Each weight w_i is the contribution of feature x_i: if x_i goes up by one unit and nothing else changes, the prediction changes by w_i. Different choices of w_0 and $\mathbf{w}_{1:d}$ give different lines, planes, or hyperplanes. The number of parameters is $d+1$ (one per feature, plus the intercept).

2.2 Stacking the data into a matrix

Writing out $\hat{y}_i = w_0 + w_1 x_i$ for each of our four customers gives four equations, and Notes 02 taught us that this is one matrix–vector product. The trick is to treat the intercept as a weight on a feature that is always 1. Put a column of ones in front of the data:

$$\underbrace{\begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \hat{y}_3 \\ \hat{y}_4 \end{bmatrix}}_{\hat{\mathbf{y}}} = \underbrace{\begin{bmatrix} 1 & 25 \\ 1 & 30 \\ 1 & 40 \\ 1 & 50 \end{bmatrix}}_{X} \underbrace{\begin{bmatrix} w_0 \\ w_1 \end{bmatrix}}_{\mathbf{w}}, \qquad \mathbf{y} = \begin{bmatrix} 4 \\ 5 \\ 3 \\ 2 \end{bmatrix}.$$

Row i of X is $(1,\, x_i)$, and multiplying it by $\mathbf{w}$ gives the prediction for customer i. This is the row-wise scoring of Notes 02, Section 3.3, applied to all customers at once. For n samples and d features, the general form is

$$\hat{\mathbf{y}} = X\mathbf{w}, \qquad X = \begin{bmatrix} 1 & \mathbf{x}_1^{\mathsf{T}} \\ \vdots & \vdots \\ 1 & \mathbf{x}_n^{\mathsf{T}} \end{bmatrix} \in \mathbb{R}^{n \times (d+1)}, \quad \mathbf{w} \in \mathbb{R}^{d+1}, \quad \hat{\mathbf{y}}, \mathbf{y} \in \mathbb{R}^n.$$

X is called the *design matrix*: one row per sample, one column per feature, plus the column of ones.

> Before any computation, write the shapes: X is $n \times (d+1)$, $\mathbf{w}$ is $(d+1) \times 1$, so $X\mathbf{w}$ is $n \times 1$, which matches $\mathbf{y}$. If your shapes do not match, the formula is wrong.

Check yourself. We have 100 customers and 5 measured features. What are the shapes of X, $\mathbf{w}$, and $X^\top X$?
Answer. X is 100×6, $\mathbf{w}$ is 6×1, and $X^\top X$ is 6×6. The 6 is $d + 1$: five features plus the column of ones.

2.3 Many lines: what does "best" mean?

Here are four candidate lines for the age data (Figure 2), with the predictions each makes and the total squared error $\sum_i (y_i - \hat{y}_i)^2$.

Line (w_0, w_1)	predictions at ages 25, 30, 40, 50	errors $y_i - \hat{y}_i$	total squared error
A: (5, 0)	5, 5, 5, 5	−1, 0, −2, −3	14
B: (4, 0)	4, 4, 4, 4	0, 1, −1, −2	6
C: (3.5, 0)	3.5, 3.5, 3.5, 3.5	0.5, 1.5, −0.5, −1.5	5
D: (6, −0.08)	4, 3.6, 2.8, 2	0, 1.4, 0.2, 0	2

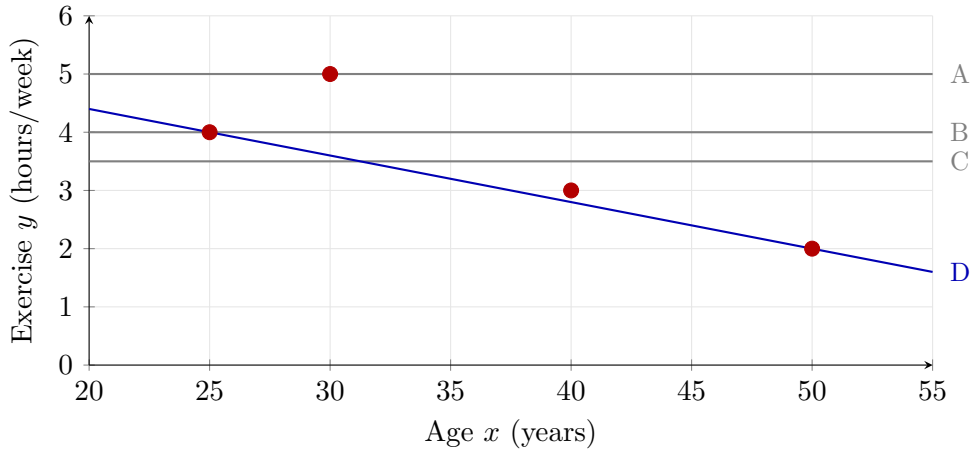


Figure 2: The four customers and the four candidate lines of the table. Line D follows the downward trend; A, B and C ignore it.

The sloped line is clearly better than the flat ones. A natural definition of "best" is therefore: *the line whose errors, taken together, are smallest.* To make this precise we need a single number that summarizes all errors. That number is the loss.

3 The loss function

> **Where this fits.** Steps 2 and 3 of the recipe: turn "best" into a single number we can minimize, and write the minimization problem down.

3.1 From residuals to a loss

For sample i the residual is

$$e_i = y_i - \hat{y}_i = y_i - w_0 - \mathbf{w}_{1:d}^\top \mathbf{x}_i, \qquad i = 1, \ldots, n.$$

Geometrically, it is the vertical gap between the observed point and the model's line. A residual can be positive or negative, so we cannot simply add them (a +2 would cancel a −2). Squaring

each residual first fixes the sign problem, and gives three common loss functions:

$$\text{least-squares error (LSE):} \quad \mathcal{L}(\mathbf{w}) = \sum_{i=1}^{n} e_i^2,$$

$$\text{mean-squared error (MSE):} \quad \mathcal{L}(\mathbf{w}) = \frac{1}{n}\sum_{i=1}^{n} e_i^2,$$

$$\text{root mean-squared error (RMSE):} \quad \mathcal{L}(\mathbf{w}) = \sqrt{\frac{1}{n}\sum_{i=1}^{n} e_i^2}.$$

The loss depends on $\mathbf{w}$ because each e_i does. Training means finding the $\mathbf{w}$ that makes it small.

These three are the same up to a positive constant $(1/n)$ and a square root, and neither operation changes *which* $\mathbf{w}$ gives the smallest value: if $a < b$ then $a/n < b/n$ and $\sqrt{a} < \sqrt{b}$. So all three share exactly one *minimizer* (the $\mathbf{w}$ with the smallest loss), and we may pick whichever is most convenient. MSE and RMSE are easier to interpret (RMSE is in the units of y, so "about half an hour per week" means something). The LSE is easiest to differentiate, and we multiply it by $\frac{1}{2}$ so that a factor 2 produced by the derivative cancels:

$$\boxed{\mathcal{L}(\mathbf{w}) = \frac{1}{2}\sum_{i=1}^{n}\left(y_i - w_0 - \mathbf{w}_{1:d}^{\mathsf{T}}\mathbf{x}_i\right)^2}$$

The $\frac{1}{2}$ is purely a convenience and does not change the minimizer.

Why squares, and not absolute values? Squaring is not the only way to remove signs: $|e_i|$ would work too. Squares are the standard choice for three reasons. They punish a few big mistakes more than many small ones (an error of 4 costs sixteen times an error of 1). They are smooth, so we can differentiate them everywhere, whereas $|e|$ has a kink at 0. And, as we are about to see, they lead to a clean formula (Figure 3). A further justification (squared error is the right loss if the noise n is Gaussian) comes in the probability part of the course.

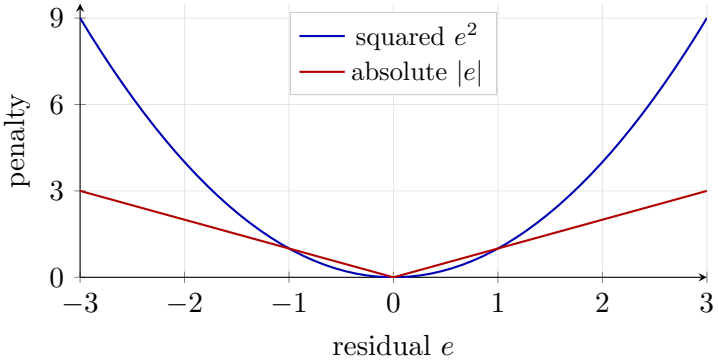


Figure 3: Two ways to turn a residual into a cost. The square is smooth at 0 and grows quickly: an error of 3 costs nine times an error of 1, versus three times for the absolute value.

3.2 The same loss in vector form

Stack the residuals into a vector $\mathbf{e} = (e_1, \ldots, e_n)^{\mathsf{T}}$. Then $\sum_i e_i^2 = \mathbf{e}^{\mathsf{T}}\mathbf{e} = \|\mathbf{e}\|_2^2$, and the residual vector is

$$\mathbf{e} = \mathbf{y} - \hat{\mathbf{y}} = \mathbf{y} - X\mathbf{w}.$$

So the loss is

$$\mathcal{L}(\mathbf{w}) = \tfrac{1}{2}(\mathbf{y} - X\mathbf{w})^{\mathsf{T}}(\mathbf{y} - X\mathbf{w}) = \tfrac{1}{2}\|\mathbf{y} - X\mathbf{w}\|_2^2.$$

This is the squared Euclidean distance (Notes 01) between the observed outputs and the model's outputs. Fitting a linear model means finding the $\mathbf{w}$ for which $X\mathbf{w}$ is as close as possible to $\mathbf{y}$. The optimization problem is

$$\underset{\mathbf{w}\in\mathbb{R}^{d+1}}{\text{minimize}} \quad \tfrac{1}{2}\|\mathbf{y} - X\mathbf{w}\|_2^2.$$

It has no constraints: every $\mathbf{w}$ is allowed.

In AI. Squared error is the default loss for regression in machine learning: forecasting demand, estimating the remaining life of a battery from its charging history, or scoring driving quality from sensor data. The squared-norm loss $\|\mathbf{y} - \hat{\mathbf{y}}\|_2^2$ is also the building block for training neural networks on numerical targets.

4 The closed-form solution

> **Where this fits.** Step 4 of the recipe, and the main result of the lecture: the best parameters in a single formula.

4.1 The intuition: a bowl

Fix the data and think of $\mathcal{L}$ as a function of (w_0, w_1). For each pair there is one loss value, so the loss is a surface above the (w_0, w_1) plane. Because the loss is a sum of squares of quantities that are linear in $\mathbf{w}$, the surface is a smooth bowl with a single lowest point. At the bottom of a bowl the ground is flat: no direction is downhill. In calculus terms, the slope of the loss in every direction is zero. We find the bottom by asking where the slope is zero, which is the same idea as setting $f'(x) = 0$ for a one-variable function.

Figure 4 draws the bowl as a contour map, like the height lines on a hiking map: every point on one curve has the same loss, and the centre is the bottom. The black dot is the best line (we compute it below); the labelled squares are the candidate lines A–D from Section 2, each sitting on the contour that matches its loss.

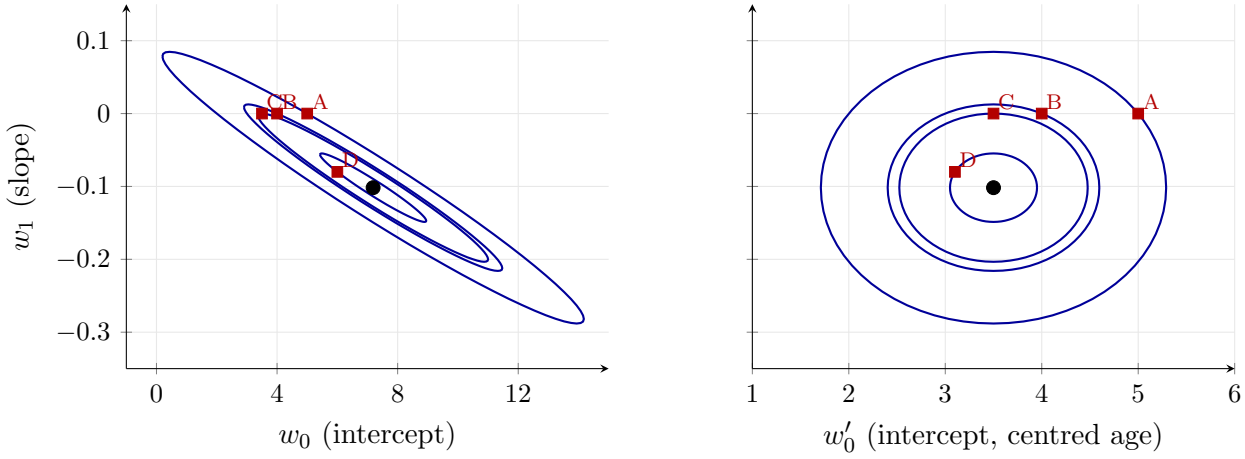


Figure 4: Contours of $\mathcal{L}(w_0, w_1) = \frac{1}{2}\sum_i (y_i - w_0 - w_1 x_i)^2$ for the age data. The black dot is the minimizer; the squares are lines A–D with losses 7, 3, 2.5, 1 (half their total squared errors). *Left:* raw ages give a long thin valley. *Right:* the same loss after subtracting the mean age 36.25 from every age, which makes the bowl much rounder (the intercept is re-defined as the prediction at the mean age, $w_0' = w_0 + 36.25\, w_1$).

Two things are worth noticing in the figure. First, there is a single lowest point and no other dips, which is what makes "set the slope to zero" sufficient. Second, the left panel is a narrow valley because ages are large numbers: moving the slope w_1 by 0.1 shifts predictions by several hours, while moving w_0 by 0.1 shifts them by only 0.1. Such a valley is numerically awkward, and we come back to it in Section 8.

4.2 Derivation

Start from $\mathcal{L}(\mathbf{w}) = \frac{1}{2}(\mathbf{y} - X\mathbf{w})^\mathsf{T}(\mathbf{y} - X\mathbf{w})$ and multiply out, using the rule $(X\mathbf{w})^\mathsf{T} = \mathbf{w}^\mathsf{T}X^\mathsf{T}$ for the transpose of a product:

$$\mathcal{L}(\mathbf{w}) = \tfrac{1}{2}\big(\mathbf{y}^\mathsf{T}\mathbf{y} - \mathbf{y}^\mathsf{T}X\mathbf{w} - \mathbf{w}^\mathsf{T}X^\mathsf{T}\mathbf{y} + \mathbf{w}^\mathsf{T}X^\mathsf{T}X\mathbf{w}\big).$$

The two middle terms are 1×1 (scalars), and a scalar equals its own transpose, so $\mathbf{y}^\mathsf{T}X\mathbf{w} = (\mathbf{y}^\mathsf{T}X\mathbf{w})^\mathsf{T} = \mathbf{w}^\mathsf{T}X^\mathsf{T}\mathbf{y}$. They are equal, which gives

$$\mathcal{L}(\mathbf{w}) = \tfrac{1}{2}\big(\mathbf{y}^\mathsf{T}\mathbf{y} - 2\mathbf{w}^\mathsf{T}X^\mathsf{T}\mathbf{y} + \mathbf{w}^\mathsf{T}X^\mathsf{T}X\mathbf{w}\big).$$

Now we need the slope of $\mathcal{L}$ with respect to $\mathbf{w}$. With a vector of unknowns, the slope is the *gradient*: the vector of partial derivatives $\nabla\mathcal{L} = [\partial\mathcal{L}/\partial w_0,\ \ldots,\ \partial\mathcal{L}/\partial w_d]^\mathsf{T}$. It points in the direction of steepest increase and is the zero vector at a flat spot. Each of the three terms of $\mathcal{L}$ has a one-variable twin whose derivative we already know, and the vector rule has the same shape.

- $\mathbf{y}^\mathsf{T}\mathbf{y}$ does not involve $\mathbf{w}$, so its gradient is $\mathbf{0}$ (a constant has zero derivative).
- $\mathbf{w}^\mathsf{T}(X^\mathsf{T}\mathbf{y})$ is a fixed vector $\mathbf{a} = X^\mathsf{T}\mathbf{y}$ dotted with $\mathbf{w}$. This is the twin of $\frac{d}{dw}(aw) = a$, and the rule is $\nabla(\mathbf{a}^\mathsf{T}\mathbf{w}) = \mathbf{a}$. So the gradient is $X^\mathsf{T}\mathbf{y}$. (Check with two unknowns: for $3w_1 + 5w_2$, the partials are 3 and 5, which is the vector $\mathbf{a} = (3, 5)^\mathsf{T}$.)
- $\mathbf{w}^\mathsf{T}(X^\mathsf{T}X)\mathbf{w}$ is the twin of $\frac{d}{dw}(pw^2) = 2pw$, and the rule is $\nabla(\mathbf{w}^\mathsf{T}P\mathbf{w}) = 2P\mathbf{w}$, so the gradient is $2X^\mathsf{T}X\mathbf{w}$. This rule needs P to be symmetric (for a general P it is $(P + P^\mathsf{T})\mathbf{w}$). Here $P = X^\mathsf{T}X$ is a Gram matrix, which is always symmetric (Notes 02, Section 5.2), so the simple form applies.

Putting these together,

$$\nabla\mathcal{L}(\mathbf{w}) = \tfrac{1}{2}\big(-2X^\mathsf{T}\mathbf{y} + 2X^\mathsf{T}X\mathbf{w}\big) = X^\mathsf{T}X\mathbf{w} - X^\mathsf{T}\mathbf{y}.$$

The $\frac{1}{2}$ cancelled the 2, as promised. Setting the gradient to zero gives the *normal equations*, and solving them gives the answer:

$$X^\mathsf{T}X\,\mathbf{w} = X^\mathsf{T}\mathbf{y} \qquad \Longrightarrow \qquad \boxed{\mathbf{w}^\star = (X^\mathsf{T}X)^{-1}X^\mathsf{T}\mathbf{y}}$$

This $\mathbf{w}^\star$ is the *least-squares solution*. It minimizes the LSE, MSE, and RMSE simultaneously.

In the language of Notes 02, Section 8.6, the matrix $(X^\mathsf{T}X)^{-1}X^\mathsf{T}$ is the *left pseudo-inverse* of the tall matrix X. When $X\mathbf{w} = \mathbf{y}$ has no exact solution (four points almost never lie on one line), the pseudo-inverse returns the $\mathbf{w}$ that comes closest in squared error. This is exactly what Notes 02 promised: ordinary least squares is the left-inverse at work.

4.3 Worked example: the age data

For our four customers,

$$X = \begin{bmatrix} 1 & 25 \\ 1 & 30 \\ 1 & 40 \\ 1 & 50 \end{bmatrix}, \qquad \mathbf{y} = \begin{bmatrix} 4 \\ 5 \\ 3 \\ 2 \end{bmatrix}.$$

Entry by entry, $X^\top X$ contains $n = 4$, the sum of ages $25 + 30 + 40 + 50 = 145$, and the sum of squared ages $625 + 900 + 1600 + 2500 = 5625$; and $X^\top \mathbf{y}$ contains $\sum y_i = 14$ and $\sum x_i y_i = 100 + 150 + 120 + 100 = 470$:

$$X^\top X = \begin{bmatrix} 4 & 145 \\ 145 & 5625 \end{bmatrix}, \qquad X^\top \mathbf{y} = \begin{bmatrix} 14 \\ 470 \end{bmatrix}.$$

The determinant is $4 \cdot 5625 - 145^2 = 22500 - 21025 = 1475$, so

$$(X^\top X)^{-1} = \frac{1}{1475} \begin{bmatrix} 5625 & -145 \\ -145 & 4 \end{bmatrix},$$

and

$$\mathbf{w}^\star = \frac{1}{1475} \begin{bmatrix} 5625 \cdot 14 - 145 \cdot 470 \\ -145 \cdot 14 + 4 \cdot 470 \end{bmatrix} = \frac{1}{1475} \begin{bmatrix} 10600 \\ -150 \end{bmatrix} = \begin{bmatrix} 424/59 \\ -6/59 \end{bmatrix} \approx \begin{bmatrix} 7.186 \\ -0.1017 \end{bmatrix}.$$

The best line is $\hat{y} = 7.186 - 0.1017\,x$. Its predictions are $\hat{y} = (424 - 6x)/59$, which gives 4.644, 4.136, 3.119, 2.102 at the four ages, so the residuals are $(-38, 51, -7, -6)/59$. The total squared error is $(38^2 + 51^2 + 7^2 + 6^2)/59^2 = 4130/3481 \approx 1.186$. Compare this with the table in Section 2: the best line beats the line $(6, -0.08)$ that we guessed (total squared error 2), and no other line can do better. The RMSE is $\sqrt{1.186/4} \approx 0.54$ hours: typical predictions are off by about half an hour.

A cross-check with the familiar formula. For one feature the matrix formula must agree with the slope formula from school statistics, $w_1 = \sum_i (x_i - \bar{x})(y_i - \bar{y}) / \sum_i (x_i - \bar{x})^2$ and $w_0 = \bar{y} - w_1 \bar{x}$. Here $\bar{x} = 36.25$, $\bar{y} = 3.5$, $\sum_i (x_i - \bar{x})(y_i - \bar{y}) = -37.5$ and $\sum_i (x_i - \bar{x})^2 = 368.75$, so $w_1 = -37.5/368.75 = -0.1017$ and $w_0 = 3.5 + 0.1017 \cdot 36.25 = 7.186$. It agrees. The same numbers explain the right panel of Figure 4: once the ages are centred, $\sum_i (x_i - \bar{x}) = 0$, so the off-diagonal entry 145 of $X^\top X$ becomes 0, giving $X^\top X = \mathrm{diag}(4, 368.75)$ and a bowl that is no longer tilted.

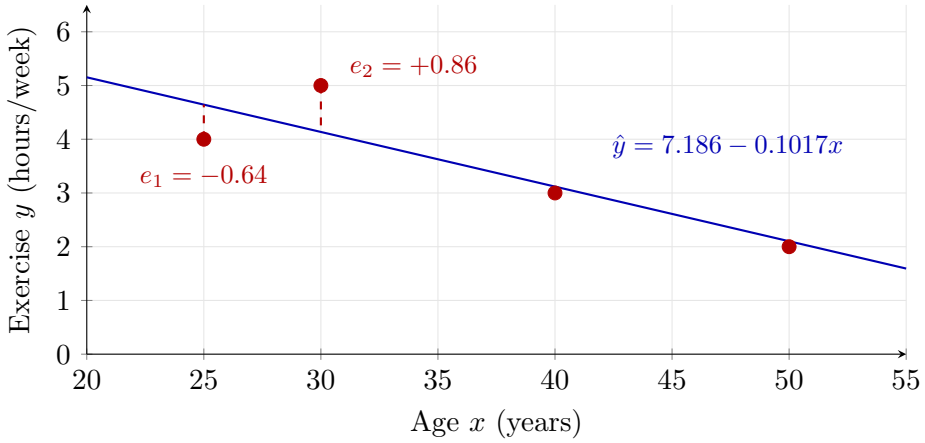


Figure 5: The least-squares line for the four customers. Dashed segments are the residuals; the line is the one that makes the sum of their squares smallest.

How to read the coefficients. The slope -0.1017 says that, on this data, each extra year of age goes with 0.10 fewer hours of exercise per week (about six minutes). The intercept 7.19 is the prediction at age zero, which is meaningless as a statement about newborns. It is simply where the line meets the vertical axis. This reminds us that a model is trustworthy only near the data it was fitted on: the same line predicts 0.07 hours at age 70 and a *negative* -0.95 hours at age 80.

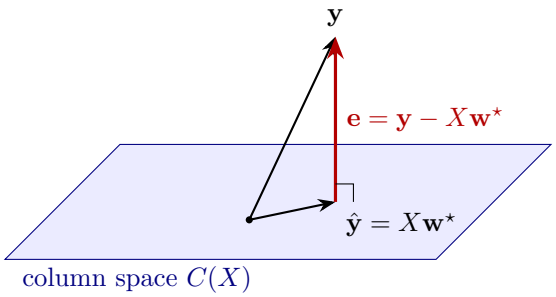
Check yourself. Suppose we record age in months instead of years, so every entry in the age column is multiplied by 12. What happens to w_1 and to the predictions?
Answer. w_1 is divided by 12 (a "per month" slope is a twelfth of a "per year" slope), w_0 is unchanged, and the predictions are identical. The same line is described in different units.

4.4 What the answer means geometrically

The normal equations can be rewritten as

$$X^{\mathsf{T}}(\mathbf{y} - X\mathbf{w}^\star) = \mathbf{0} \qquad \text{that is,} \qquad X^{\mathsf{T}}\mathbf{e} = \mathbf{0}.$$

Row j of X^{T} is column j of X, so this says: *the residual vector is orthogonal to every column of X*. Recall from Notes 02 that the set of all vectors $X\mathbf{w}$ is the column space $C(X)$ of X. The fitted vector $\hat{\mathbf{y}} = X\mathbf{w}^\star$ is therefore the point in $C(X)$ closest to $\mathbf{y}$, namely the orthogonal projection of $\mathbf{y}$ onto $C(X)$, and the error is what is left over. (It is closest because, for any other point $\mathbf{p}$ in $C(X)$, orthogonality and Pythagoras give $\|\mathbf{y} - \mathbf{p}\|^2 = \|\mathbf{e}\|^2 + \|\hat{\mathbf{y}} - \mathbf{p}\|^2 \geq \|\mathbf{e}\|^2$.) The picture below is a cartoon: for our data $C(X)$ is a two-dimensional plane inside $\mathbb{R}^4$, drawn here as a plane in three dimensions.



Two practical consequences follow because the first column of X is all ones.

- Orthogonality to the column of ones means $\sum_i e_i = 0$: the residuals add up to zero. In our example $-38 + 51 - 7 - 6 = 0$. The other row of $X^{\mathsf{T}}\mathbf{e} = \mathbf{0}$ checks the age column: $(25 \cdot (-38) + 30 \cdot 51 + 40 \cdot (-7) + 50 \cdot (-6))/59 = 0$.
- The fitted line passes through the point of averages $(\bar{x}, \bar{y}) = (36.25,\ 3.5)$. Check: $7.186 - 0.1017 \cdot 36.25 = 3.5$.

These are quick sanity checks for any least-squares fit you compute by hand.

4.5 Why it is the minimum, and when the inverse exists

It is a true minimum. Setting the gradient to zero finds a flat spot, which could in principle be a maximum or a saddle. Here it is always a minimum. Recall the one-variable case: $f(w) = \frac{1}{2}pw^2$ has a flat spot at $w = 0$, and it is a minimum exactly when the second derivative p is positive. The matrix version of the second derivative is the *Hessian*, the matrix of all second partial derivatives. For our loss it is $X^{\mathsf{T}}X$, and the matrix version of "$p \geq 0$" is that, for any vector $\mathbf{v}$,

$$\mathbf{v}^{\mathsf{T}}X^{\mathsf{T}}X\mathbf{v} = \|X\mathbf{v}\|_2^2 \ \geq\ 0.$$

The loss curves upward in every direction, which is the bowl of Figure 4, so the flat spot is the global minimum.

When can we invert $X^\top X$? Suppose $X^\top X\mathbf{v} = \mathbf{0}$. Multiplying by $\mathbf{v}^\top$ gives $\|X\mathbf{v}\|_2^2 = 0$, hence $X\mathbf{v} = \mathbf{0}$. If the columns of X are linearly independent (Notes 02, Section 4.2), the only such $\mathbf{v}$ is $\mathbf{0}$, and then $X^\top X$ is invertible and the minimizer is unique. If a column is redundant, $X^\top X$ is singular, there is a whole line or plane of equally good $\mathbf{w}$'s, and the formula $(X^\top X)^{-1}$ breaks down. Two things cause this:

- *Too little data.* We need $n \geq d + 1$, at least as many samples as parameters. With a single customer you cannot determine a line.
- *Redundant features.* If one feature is a copy or a combination of the others (for example, height in centimetres and height in metres), its column adds nothing and $X^\top X$ is singular.

> Before you run least squares, ask: do I have at least $d + 1$ samples, and is any feature just a rescaled copy of another? If so, remove it. (Later in the course, regularization gives another fix.)

Check yourself. A 1000×50 feature matrix has all 50 features distinct, but feature 50 equals twice feature 3. Does $(X^\top X)^{-1}$ exist?
Answer. No. Column 50 is $2\times$ column 3, so the columns are linearly dependent, X does not have full column rank, and $X^\top X$ is singular.

5 More features: age and sleep

> **Where this fits.** Shows that the closed form needs no change for more than one feature: we add a column to X and nothing else.

The company now records the average hours of sleep per night as a second feature.

Age x_1	Sleep x_2 (hours/night)	Exercise y (hours/week)
25	7	4
30	6	5
40	6	3
50	5	2

Intuition. With two inputs, every customer is a point in a three-dimensional space (age, sleep, exercise), and the model is a flat sheet (a plane) that we try to push as close to those points as possible. Nothing about the method changes. We just add a column. Figure 6 shows the picture.

The maths. The model is $\hat{y} = w_0 + w_1 x_1 + w_2 x_2$, and

$$X = \begin{bmatrix} 1 & 25 & 7 \\ 1 & 30 & 6 \\ 1 & 40 & 6 \\ 1 & 50 & 5 \end{bmatrix}, \qquad \mathbf{w} = \begin{bmatrix} w_0 \\ w_1 \\ w_2 \end{bmatrix}, \qquad \mathbf{y} = \begin{bmatrix} 4 \\ 5 \\ 3 \\ 2 \end{bmatrix}.$$

The formula $\mathbf{w}^\star = (X^\top X)^{-1} X^\top \mathbf{y}$ is unchanged. Here

$$X^\top X = \begin{bmatrix} 4 & 145 & 24 \\ 145 & 5625 & 845 \\ 24 & 845 & 146 \end{bmatrix}, \qquad X^\top \mathbf{y} = \begin{bmatrix} 14 \\ 470 \\ 86 \end{bmatrix},$$

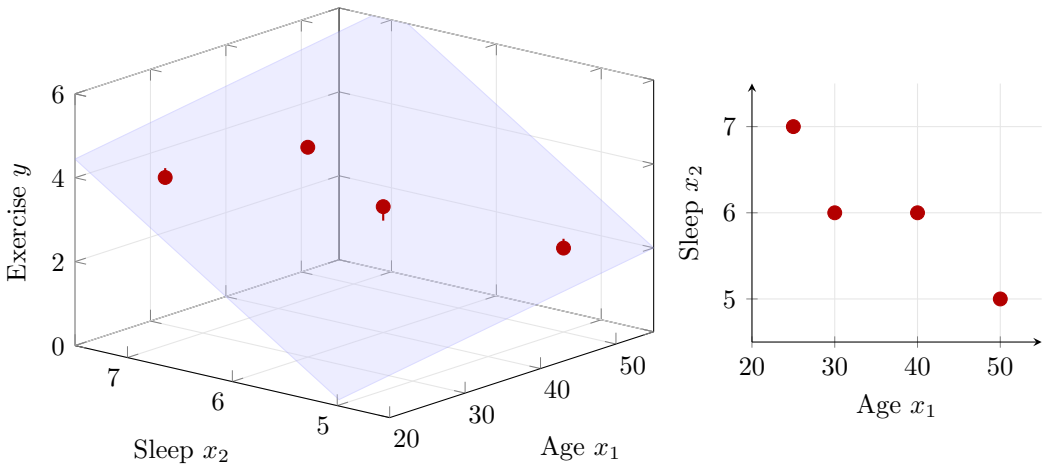


Figure 6: *Left:* the four customers as points in (age, sleep, exercise) space and the fitted plane (red segments join each point to the plane; they are the residuals, which are small here). *Right:* the same customers seen from above, looking only at the two inputs. They lie almost on a line (older customers sleep less), which is the root of the warning below.

and solving the three normal equations gives

$$\mathbf{w}^\star = \begin{bmatrix} 200/9 \\ -2/9 \\ -16/9 \end{bmatrix} \approx \begin{bmatrix} 22.22 \\ -0.222 \\ -1.778 \end{bmatrix}.$$

You can verify this by substitution. The first normal equation reads $4 \cdot \frac{200}{9} - 145 \cdot \frac{2}{9} - 24 \cdot \frac{16}{9} = \frac{800-290-384}{9} = 14$, which matches 14 on the right. (Rows two and three give 470 and 86.) The predictions are $\hat{y} = (38,\ 44,\ 24,\ 20)/9$ and the residuals are $(-2,\ 1,\ 3,\ -2)/9$, with total squared error $2/9 \approx 0.22$, much smaller than 1.19 with age alone. A customer aged 35 who sleeps 6.5 hours is predicted to exercise $22.22 - 0.222 \cdot 35 - 1.778 \cdot 6.5 \approx 2.89$ hours.

A warning that matters. The sleep weight is *negative*: more sleep goes with less exercise. Is that real? Almost certainly not. Age and sleep move together in our data (their correlation is about -0.92: older customers sleep less), and the model cannot separate their effects. A weight w_i means "the change in prediction when x_i rises by one unit *and all other features stay fixed*", and the data contain almost no cases that vary one feature while holding the other still. Two other cautions apply. Adding a feature can never increase the training error (the one-feature model is the special case $w_2 = 0$), so a smaller training error alone is no evidence that the new feature is useful. And we are fitting three parameters to four points, so there is very little left over to guard against fitting noise. We return to this in Section 7.

In AI. Real regression problems have many features. A battery model might use cycle count, average temperature, and charging rate; ride pricing might use demand, time of day, and weather. The algebra is exactly what you have just seen, with a wider X. Interpreting individual weights, however, is reliable only when features are not strongly redundant.

6 Polynomial regression: bending the line

Where this fits. Shows that the same closed form fits curves, not just lines, by changing the *columns* of X and leaving the method untouched. It also sets up the over-fitting problem of Section 7.

6.1 When a line is not enough

A line has one fixed slope, so it cannot represent a relationship that rises and then falls, or levels off. Suppose exercise rises through the twenties and thirties and then declines: no straight line captures that. Curves are needed, and a polynomial is the simplest curve.

6.2 The trick: new features from old

For a single feature x, a polynomial model of degree M is

$$\hat{f}(x, \mathbf{w}) = w_0 + w_1 x + w_2 x^2 + \cdots + w_M x^M = \sum_{i=0}^{M} w_i x^i.$$

It has $M + 1$ parameters. The degree M is chosen by us and not learned from the data; such a setting is called a *hyper-parameter* (like k in kNN). It controls the flexibility of the model. For $M = 1$ we recover the line, and $M = 0$ is a constant (for which the best constant is simply the average of the y_i; for our data, 3.5).

Here is the key observation. Define a *feature map*

$$g(x) = \left[1,\ x,\ x^2,\ \ldots,\ x^M\right]^{\mathsf{T}}.$$

Then the model is

$$\hat{f}(x, \mathbf{w}) = g(x)^{\mathsf{T}} \mathbf{w},$$

which is a linear model whose "features" are powers of x. The model is curved in x but *linear in the parameters* $\mathbf{w}$, because each w_i appears only multiplied by a number and added up. That is the only thing the least-squares derivation needed. The formula never used the fact that the columns of X were the raw measurements. For this reason polynomial regression is also called *linear-in-parameters* regression, even though it is often called "non-linear regression".

Figure 7 shows what is going on. The data on the left lie on a curve, so no line fits them. If we plot the same y values against the new feature $z = x^2$ instead, they lie close to a straight line, and fitting a straight line is exactly what linear regression does. A polynomial fit is a *line in the space of the new features*, seen back in the original coordinates as a curve.

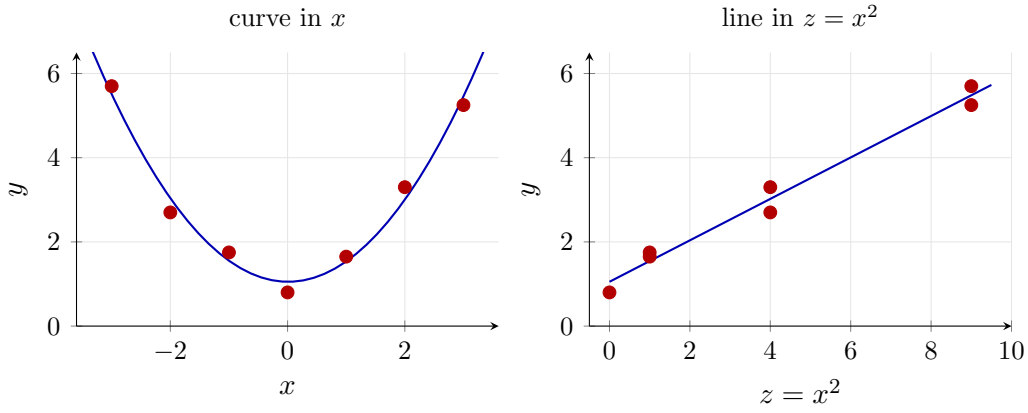


Figure 7: The feature-map idea, with illustrative made-up data. The same seven points and the same fitted model are drawn against x (a curve) and against $z = x^2$ (a straight line). The model is linear in its parameters (w_0, w_2) even though it is curved in x.

For n data points the design matrix has one row $g(x_i)^\mathsf{T}$ per sample:

$$X = \begin{bmatrix} 1 & x_1 & x_1^2 & \cdots & x_1^M \\ 1 & x_2 & x_2^2 & \cdots & x_2^M \\ \vdots & \vdots & \vdots & & \vdots \\ 1 & x_n & x_n^2 & \cdots & x_n^M \end{bmatrix}, \qquad \mathbf{e} = \mathbf{y} - X\mathbf{w}, \qquad \mathbf{w}^\star = (X^\mathsf{T}X)^{-1}X^\mathsf{T}\mathbf{y}.$$

We have solved this problem already. Nothing new is needed except building a wider X. The feature map g can contain *any* fixed functions of x, for example $[1,\ 1/x,\ \sin(2\pi x),\ e^x]^\mathsf{T}$, and the same formula still applies.

> "Linear" in linear regression means linear in the *parameters* $\mathbf{w}$, not in the input. $y = w_0 + w_1 \sin x + w_2 e^x$ is linear regression. $y = w_0 + e^{w_1 x}$ is not, because w_1 sits inside the exponential.

6.3 Our age data with $M = 0, 1, 2, 3$

We fit polynomials of increasing degree to the four customers by building X with columns $1, x, x^2, \ldots$ and solving the normal equations. The results are:

M	parameters	fitted polynomial	total squared error
0	1	3.5	5.0
1	2	$7.186 - 0.1017x$	1.19
2	3	$1.819 + 0.2043x - 0.004070x^2$	0.91
3	4	$-59 + 5.403x - 0.1470x^2 + 0.001267x^3$	0 (exact)

Training error falls steadily and reaches *exactly zero* at $M = 3$. The reason is simple counting: a cubic has four parameters, there are four data points, and the four equations $X\mathbf{w} = \mathbf{y}$ can be solved exactly, since X is then square and invertible (its columns are the powers $1, x, x^2, x^3$ of the same four distinct ages, and such a matrix is always invertible). Zero error looks like a triumph, so look at the curves.

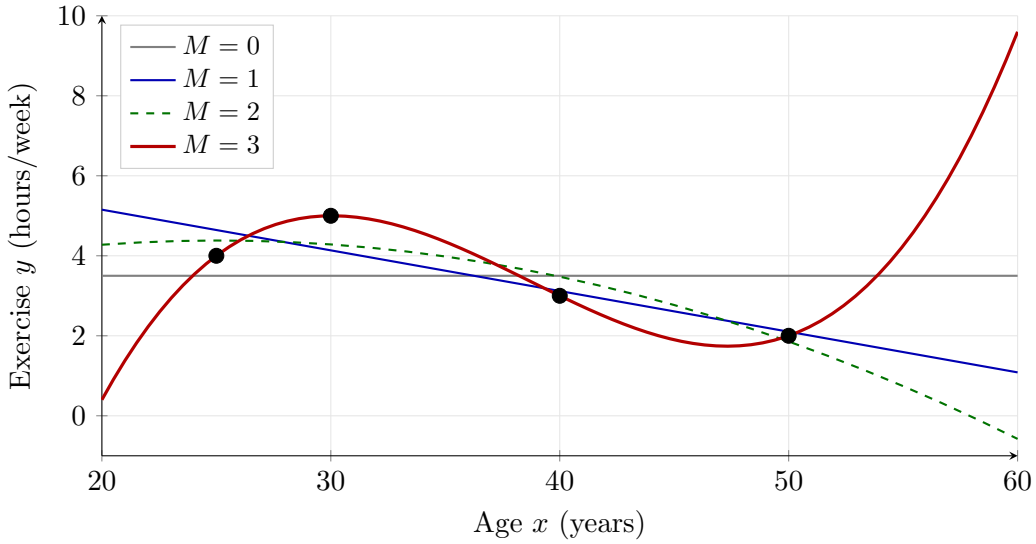


Figure 8: Four polynomial fits to the same four points. The cubic passes through every point and then swings to 9.6 hours at age 60.

The cubic claims that people aged 35 exercise 4.35 hours (more than anyone in the data), that 45-year-olds exercise 1.9, and that 60-year-olds exercise 9.6 hours a week. No one believes the last number.

	age 20	age 35	age 45	age 60
$M = 1$	5.15	3.63	2.61	1.08
$M = 2$	4.28	3.98	2.77	−0.58
$M = 3$	0.40	4.35	1.90	9.60

6.4 A bigger example from the lecture: fitting a sine wave

The lecture illustrates the same effect with a cleaner experiment, taken from Bishop's *Pattern Recognition and Machine Learning*, Section 1.1. The true process is $f(x) = \sin(2\pi x)$ on $[0, 1]$; we observe $n = 10$ noisy points $y = f(x) + n$ and fit polynomials of degree $M = 0, 1, 3, 9$. The curve for $M = 0$ is a flat line and for $M = 1$ a tilted line, and neither can follow the wave. The cubic $M = 3$ follows it well. At $M = 9$ the curve passes through every point exactly (ten parameters, ten points, zero training error) but oscillates wildly between them and bears little resemblance to the sine wave. The fitted coefficients in the lecture show what is going on:

	$M = 0$	$M = 1$	$M = 3$	$M = 9$
w_0	0.19	0.82	0.31	0.35
w_1		−1.27	7.99	232.37
w_2			−25.43	−5321.83
w_3			17.37	48568.31
$w_4, \ldots, w_9$				−231639.30, 640042.26, …, 125201.43

Figure 9 re-creates the experiment with our own simulated points, so its curves differ in detail from the lecture's but show the same behaviour.

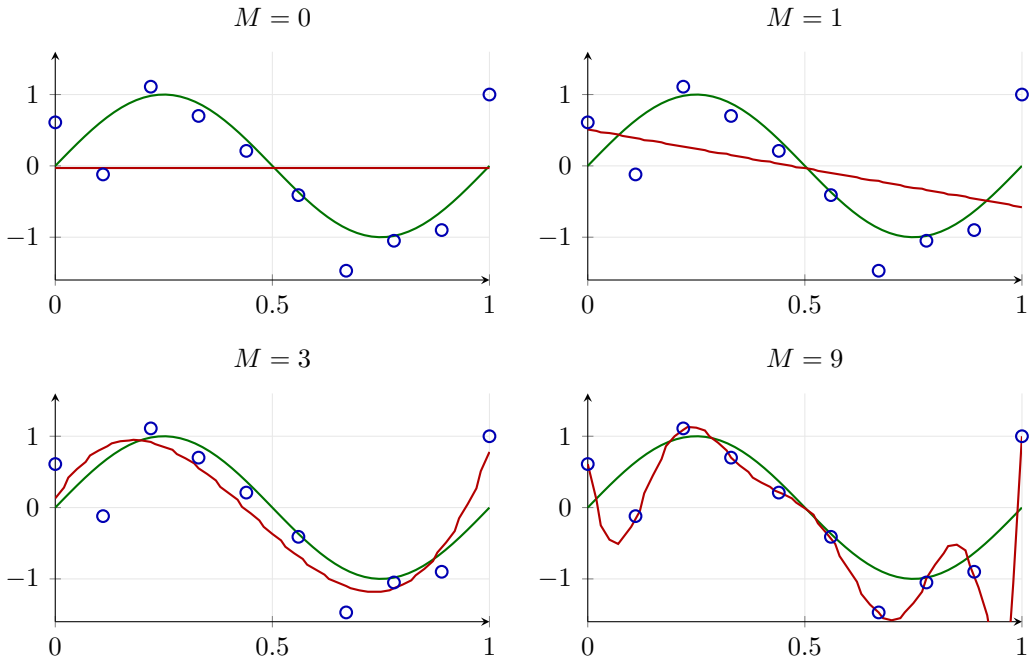


Figure 9: Fitting 10 noisy samples of $\sin(2\pi x)$ with polynomials of degree $M = 0, 1, 3, 9$. Green: the true function. Blue circles: the data. Red: the fitted polynomial (simulated data, noise standard deviation 0.3). The $M = 9$ curve hits every circle and swings wildly in between.

The lecture's own coefficients, in the table above, tell the same story. The coefficients explode in size as M grows. To pass through every noisy point the curve needs huge positive and negative terms that cancel in just the right places, and such a curve is violent between the points.

7 Underfitting and overfitting

Where this fits. The price of flexibility. The degree M of a polynomial is ours to choose, and this section explains how to choose it and why training error alone cannot.

7.1 Two ways to be wrong

- *Underfitting*: the model is too simple to capture the pattern. Its error is large on the training data *and* on new data. (The line for the sine wave, $M = 0$ or 1.)
- *Overfitting*: the model is flexible enough to fit the noise as well as the pattern. Its training error is tiny but its error on new data is large. (The $M = 9$ curve, and the cubic in our age example.)

The point of regression is to predict for customers we have *not* seen. The quantity we care about is therefore performance on new data (*generalization*), not training error. Training error always falls as the model gets more flexible, so it is useless for choosing M: it would pick the most complex model every time.

A model that has memorized the noise has nothing to say about new noise. In the cubic, four parameters were tuned to four points, so all of the noise in those points was absorbed into the curve. In the sine example, 10 parameters were tuned to 10 points.

7.2 Remedy 1: validate on held-out data

We cannot compare models by how well they fit data they have already seen. Instead, hold some data back. Fit the model on a *training set*; measure the error on a *validation set* the model never saw; and choose the degree M with the smallest validation error. In the lecture's sine experiment the training error (RMS error) falls all the way to zero at $M = 9$, while the validation error falls until about $M = 3$, stays flat to $M = 8$, and then shoots up at $M = 9$. Figure 10 repeats this with our simulated data: training error keeps falling to zero, while the validation error bottoms out at $M = 3$–4 and then rises.

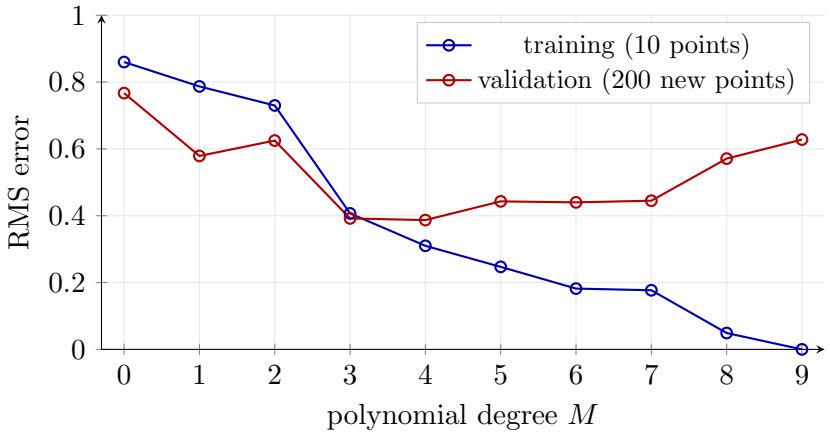


Figure 10: Training and validation RMS error against polynomial degree M for the simulated sine data of Figure 9. Training error can only fall as M grows; validation error is lowest near $M = 3$–4. (One simulated draw, so the curve is noisy.)

We can imitate this with our four customers. Four points are too few for a real validation set, so we use leave-one-out: hold out one customer, fit on the other three, and measure the error on the held-out person. Repeating for each of the four customers gives, for the root mean-squared error of the held-out predictions:

$M = 0$ (constant)	$M = 1$ (line)	$M = 2$ (parabola)
1.49	**1.06**	3.57

($M = 3$ cannot be tried: three training points do not determine four parameters.) The line wins. The parabola has the smaller training error, but it generalizes worst: when a point at the edge is held out, the parabola extrapolates badly. With only four customers these numbers are noisy, but the lesson is correct, and it is the same lesson as in the lecture.

7.3 Remedy 2: more data

An overfitted model has too much freedom for the amount of data. The sine experiment with $M = 9$ and $n = 10$ points oscillates wildly; with $n = 15$ it settles down; with $n = 100$ the same degree-9 polynomial lies almost on top of the true $\sin(2\pi x)$ (Figure 11). More data constrains the curve so that it cannot wander between points, and the same model can be a good choice with enough data.

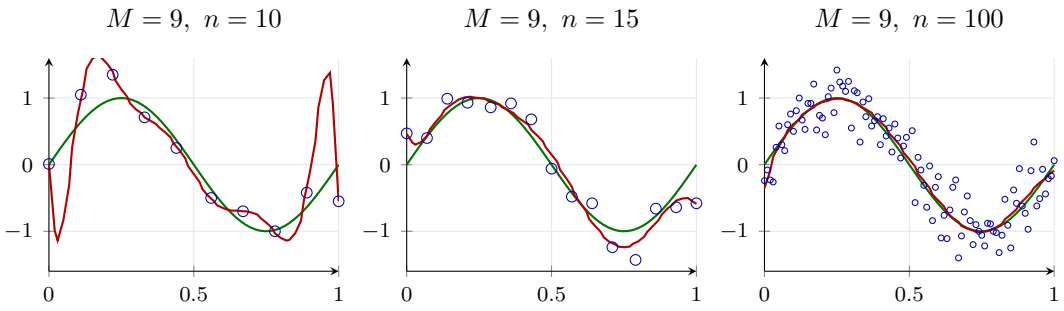


Figure 11: The same degree-9 model fitted to $n = 10$, 15 and 100 simulated samples. With little data it oscillates; with a hundred points it settles on the true curve (green).

7.4 Remedy 3: regularization (next topic)

When more data is not available, we can penalize large coefficients directly, since the table in Section 6.4 shows that overfit models have huge ones. Adding a term to the loss that grows with $\|\mathbf{w}\|$ keeps the coefficients small, which keeps the curve tame. This is regularization, and it is where the lecture goes next. (It also repairs the singular $X^{\mathsf{T}}X$ problem mentioned above.)

In AI. Overfitting is the central practical danger in machine learning, from a polynomial with ten coefficients to a neural network with billions of weights. The defence is always the same set of ideas: judge the model on data it has not seen, give it enough data, and limit its freedom.

Check yourself. You fit a degree-M polynomial to n points with distinct x-values. For which M can you guarantee zero training error, and is that a good sign?
Answer. For $M \geq n - 1$ (at least n parameters), since the design matrix then has full row rank and $X\mathbf{w} = \mathbf{y}$ can be solved exactly. It is not a good sign. It only says the model has enough freedom to memorize the data, noise included.

8 The closed form in practice

Where this fits. How the closed form is used in real software and in a real application: what to do differently from the hand calculation, and where the method leads.

8.1 Do not compute the inverse

The formula $(X^\top X)^{-1}X^\top \mathbf{y}$ is how we *derive and reason about* the answer; it is not how software computes it. Forming $X^\top X$ squares the sensitivity of the problem to rounding error, and explicit inverses magnify it further. Libraries solve the normal equations, or the original least-squares problem, with factorizations that are more stable. In NumPy:

```
import numpy as np
age = np.array([25, 30, 40, 50.])
y   = np.array([ 4,  5,  3,  2.])
X   = np.column_stack([np.ones(4), age])      # design matrix
w, *_ = np.linalg.lstsq(X, y, rcond=None)     # [ 7.18644, -0.10169 ]
```

This returns the same $\mathbf{w}^\star = (7.186, -0.1017)$ that we computed by hand.

8.2 Scale your features

Recall the long thin valley in the left panel of Figure 4. Its cause was that raw ages are large numbers, and the cure was the right panel: subtract the mean age. In numbers, for the straight-line fit the condition number of $X^\top X$ (a measure of how much errors in the input get magnified in the solution) drops from about 21 000 to about 92 when the age is centred. Polynomials make the problem far worse. Look at the cubic on raw ages. The columns $1,\ x,\ x^2,\ x^3$ range from 1 to over 100 000, and the matrix $X^\top X$ is so badly conditioned (its condition number is about 9×10^{13}) that a computer carrying about 16 digits loses nearly all of them. If instead we first standardize the feature, $z = (x - 36.25)/9.60$ (subtract the mean age and divide by the standard deviation), the condition number falls to about 80, and the same cubic is computed reliably. The fitted curve is the same; only the arithmetic is safer. For a design matrix with large M, standardizing is essential. The same advice applies to ordinary multi-feature regression where features have very different units (age in years, income in rupees).

8.3 Cost

Forming $X^\top X$ takes about $n(d+1)^2$ multiplications, and solving the $(d+1) \times (d+1)$ system takes about $(d+1)^3$, so doubling the number of features multiplies the solve cost by about eight. For a few hundred features the closed form is instantaneous. For millions of features (or data too large to hold in memory) it is impractical, and we turn to iterative algorithms that approach the minimum step by step. Linear regression is a model for which an analytical solution exists; most models we meet later (such as neural networks) do not have one, and the iterative route becomes the only choice.

8.4 A real-world problem: remaining useful life of a battery

One of the lecture's examples is predicting the remaining useful life (RUL) of a battery from its charging and discharging history. Each of the ideas in this lecture appears:

1. *Set-up.* Each battery gives one sample: features $\mathbf{x}$ such as number of charge cycles, average operating temperature, and charging rate; target y equal to the number of cycles it lasted from that point. The data are noisy because no two batteries are identical.

2. *Linear model.* $\hat{y} = w_0 + w_1 x_1 + w_2 x_2 + w_3 x_3$, fitted by $\mathbf{w}^\star = (X^\mathsf{T} X)^{-1} X^\mathsf{T} \mathbf{y}$. The weights say how much each factor shortens or lengthens life, subject to the caution about redundant features from Section 5.

3. *Polynomial features.* Batteries do not degrade at a constant rate, so a line in cycle count underfits. Adding x_1^2 (or a term like $\sqrt{x_1}$) bends the model while keeping the same solver.

4. *Overfitting.* With a few dozen batteries and many added features, the training error can be driven to zero while predictions for new batteries become useless. Keep whole batteries aside for validation. If measurements from the same battery appear both in training and in validation, the validation error will look better than it really is.

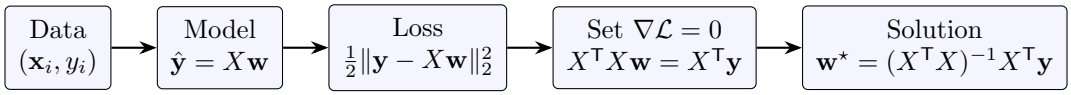
5. *Practicalities.* Standardize features, use a least-squares routine rather than an explicit inverse, and report RMSE in the units of the target ("off by about X cycles").

The same workflow serves the lecture's other examples (surge pricing, crop yield from satellite data, machine efficiency from sensor readings): only the features and the target change.

8.5 Where this leads

Linear regression is the simplest case of a pattern that runs through machine learning: a model that is linear in its parameters, a squared-error loss, and a closed form. The feature map $g(x)$ is the first hint of *representation learning*: here we choose the features by hand (powers of x), while a neural network learns them from data. The product $X\mathbf{w}$ is the computation inside every layer of a neural network (Notes 02, Section 3.3), so what you learned about how X, $\mathbf{w}$ and the loss fit together will be used repeatedly.

9 Summary map



Idea	Key formula or fact	Remember
Regression model	$y = f(\mathbf{x}) + n$, model $\hat{y} = \hat{f}(\mathbf{x}, \mathbf{w})$, error $e = y - \hat{y}$	We estimate an unknown f from noisy samples
Linear model	$\hat{\mathbf{y}} = X\mathbf{w}$, $X \in \mathbb{R}^{n \times (d+1)}$ with a column of ones	Parameters $= d + 1$; intercept is a weight on the ones column
Loss	$\mathcal{L} = \frac{1}{2}\|\mathbf{y} - X\mathbf{w}\|_2^2$	LSE, MSE, RMSE share one minimizer; $\frac{1}{2}$ is convenience
Closed form	$X^\mathsf{T}X\mathbf{w} = X^\mathsf{T}\mathbf{y} \Rightarrow \mathbf{w}^\star = (X^\mathsf{T}X)^{-1}X^\mathsf{T}\mathbf{y}$	Left pseudo-inverse; $X^\mathsf{T}\mathbf{e} = \mathbf{0}$; residuals sum to 0
When it works	X has independent columns, $n \geq d + 1$	Duplicate features make $X^\mathsf{T}X$ singular
Polynomial regression	$\hat{f} = g(x)^\mathsf{T}\mathbf{w}$, $g = [1, x, \ldots, x^M]^\mathsf{T}$	Linear in the *parameters*; same formula, wider X
Under/overfitting	M too small / too large; n points, n parameters $\Rightarrow$ zero error	Judge on validation data; more data; regularization (next)
In practice	use `lstsq`, standardize features	Do not form the inverse explicitly